AI Compiler Performance Engineering

Auto-Loops Books — ch19

Contents

1 Glow: Lightweight AI Compiler **1**

1.1 Glow HW matrix 1

1.2 Glow arch passes 4

1.3 Glow codegen 7

1.4 Glow tuning 10

1.5 Glow case study 13

1.6 Key Takeaways 16

1.7 Conclusion 17

Bibliography **19**

Index **21**

Chapter 1

Glow: Lightweight AI Compiler

Glow is the compiler in this Part that optimizes for the opposite end of the hardware spectrum from a datacenter GPU: it targets CPUs, accelerators, and memory-constrained edge devices, and it makes a deliberate bet that ahead-of-time compilation into a self-contained object file beats a runtime interpreter (Rotem *et al.*, 2018; PyTorch / Glow contributors, 2024). That bet matters for the same reason Chapter 1 gave: eliminating per-operator dispatch overhead is the lever that moves batch-1 latency, and Glow removes it at compile time by emitting one bundle with a single entry function rather than dispatching operators through a framework (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). The honest framing is that Glow is not where you optimize H100 LLM decode—it has no CUDA-graph-class GPU path—but it is exactly where you optimize decode on a Cortex-M or a small accelerator, and its design choices (graph lowering to a few primitives, operator stacking, profile-guided quantization) are the ones a residency-first engineer should recognize (Chen, 2026; Goodfellow *et al.*, 2016). This chapter treats Glow as a lightweight AOT compiler graded on bytes/token and launches/token at the edge, not as a build-tool tutorial (NVIDIA, 2024).

1.1 Glow HW matrix

Glow's support **matrix** is shaped by one design decision: rather than implement every operator on every **target**, the compiler lowers the high-level graph to a small set of linear-algebra primitives, so a new **hardware** backend only has to implement those primitives (Rotem *et al.*, 2018). That is why Glow spreads cheaply across accelerators and edge parts, and why its production sweet spot is the LLVM CPU

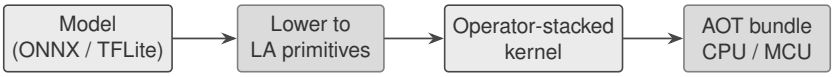


Figure 1.1: Glow lowers a model to a few linear-algebra primitives, stacks element-wise work into single-traversal kernels, and emits a self-contained AOT bundle.

backend cross-compiled to embedded cores like Arm Cortex-M, not a server GPU (PyTorch / Glow contributors, 2024).

The honest multi-hardware reading inverts the usual GPU-first story. On a **cpu** or microcontroller Glow is excellent: the AOT bundle removes interpreter and dispatch overhead and fits a fixed memory budget (Rotem *et al.*, 2018; Vellaisamy *et al.*, 2026). On a **gpu** Glow exists (an OpenCL backend) but is not the decode tool—there is no captured-launch-graph story to rival the CUDA path Chapters 16–18 described (Chen, 2026; NVIDIA, 2024). On an **npu** or custom accelerator Glow's small-primitive lowering is precisely the abstraction that makes a new backend tractable, which is its strategic value to silicon vendors (AMD, 2024b,a).

Multi-hardware delta. The same model compiles to an x86 bundle for a host and a Cortex-M bundle for an edge device, but the residency math differs by orders of magnitude: an MCU with kilobytes of SRAM cannot hold the working set an x86 cache can, so a fusion that helps on the host may not even fit on the edge target (Goodfellow *et al.*, 2016; AMD, 2024a). YiRage treats this as routing: it carries the Chapter 2 residency rules as checkable metadata and selects an edge-style AOT path or a GPU path per target rather than assuming one bundle is universally optimal (Chen and YiRage Contributors, 2026).

1.1.1 Model input and deployment forms

Glow imports models from the formats edge teams actually carry, chiefly ONNX and TFLite, and lowers them into its own graph IR before any optimization (Rotem *et al.*, 2018; PyTorch / Glow contributors, 2024). The import granularity determines what the domain-specific passes can see, exactly as it does for the datacenter compilers: an ONNX export that keeps batch normalization, bias, and activation as separate nodes leaves room for Glow's merges, while a graph that arrives pre-baked into opaque custom operators hides that structure (Chen, 2020; Davies *et al.*, 2025). For a small decoder at the edge, the import should preserve attention and MLP seams so stacking and quantization can work on real layer structure rather than on a flat list of tensors (Dao *et al.*, 2022; Rotem *et al.*, 2018). Deployment form matters too: Glow can emit bundles linked into an

application, and the choice of bundle boundary—whole model, one layer, or one bucket—sets the granularity at which the host calls into compiled code (PyTorch / Glow contributors, 2024; Davies *et al.*, 2025).

1.1.2 The primitive set as a backend contract

The small linear-algebra primitive set is the architectural bet that makes Glow's hardware matrix cheap to extend (Rotem *et al.*, 2018; AMD, 2024a). A silicon vendor adds a backend by implementing a bounded set of primitives plus the memory and thread abstractions, instead of porting every framework operator (Rotem *et al.*, 2018; Chen, 2020). For decode, the contract has two edges: high-level graphs must lower cleanly into those primitives, and the primitives must preserve enough structure for operator stacking and quantization to remain profitable (Rotem *et al.*, 2018; Vellaisamy *et al.*, 2026). When a new operator cannot lower without decomposing into many tiny primitives, the backend gets correctness but loses the single-traversal properties that edge decode depends on (Chen, 2026; Davies *et al.*, 2025). Teams evaluating an accelerator backend should therefore count how many primitives a decoder layer expands into, because that count predicts both dispatch cost and buffer traffic (Rotem *et al.*, 2018; Chen, 2026).

1.1.3 Edge decode budgets

Decode on an edge device obeys budgets that datacenter compilers ignore: SRAM and cache capacity, power envelope, binary size, and startup time are first-class constraints rather than afterthoughts (Goodfellow *et al.*, 2016; AMD, 2024a). The per-token working set of a small decoder must fit the on-chip budget or the bundle spills to slower memory on every token (Chen, 2026; Vellaisamy *et al.*, 2026). Static memory planning is therefore not a convenience on the edge; it is the mechanism that proves the bundle fits before deployment (Rotem *et al.*, 2018; Goodfellow *et al.*, 2016). Power affects the optimization objective: a schedule that saves latency by using more memory may exceed the thermal budget, so edge tuning should report energy per token alongside latency (Davies *et al.*, 2025; Chen, 2026). Teams that port a datacenter decoder to an edge device without re-deriving these budgets will discover them through field failures, not compile errors (Chen, 2020; AMD, 2024a).

1.1.4 Silicon-vendor adoption view

For a silicon vendor, Glow's lowering-to-primitives design is attractive because a new backend implements a bounded kernel set and inherits the graph optimizations



Figure 1.2: Glow's two-phase IR: a high-level node graph for domain-specific optimization, lowered via IRGen to a low-level address-only instruction IR before code generation.

(Rotem *et al.*, 2018; AMD, 2024a). The adoption decision should include the decode lens from this chapter: the vendor must prove that the primitive implementations preserve the stacking, static allocation, and precision properties the compiler assumed (Chen, 2026; Davies *et al.*, 2025). A primitive that materializes intermediates internally breaks the single-traversal contract even if it is numerically correct (Rotem *et al.*, 2018; Chen, 2026). Backend validation should therefore include decoder-shaped graphs with byte counters, not only the operator conformance suite (Chen, 2020; Vellaisamy *et al.*, 2026). Vendors that validate with decoder graphs get a credible edge-decode story; vendors that validate only with image-classification graphs inherit surprises (Davies *et al.*, 2025; Chen, 2026).

1.1.5 Bundle lifecycle and field updates

Ahead-of-time compilation moves risk from runtime to release: the bundle's schedule, memory plan, and precision are fixed when it ships, so updating a model means shipping a new bundle (Rotem *et al.*, 2018; Davies *et al.*, 2025). Edge products need an update path that can stage and validate new bundles before they replace the running one (Chen, 2020, 2026). Each bundle generation should carry a manifest with the model hash, compiler version, target, memory plan, and benchmark, so field teams can compare generations after an update (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). A field regression after an OTA update is then attributable to the bundle diff rather than to nondeterministic runtime behavior (Rotem *et al.*, 2018; Chen, 2026). The ability to roll back to the previous bundle generation is the edge counterpart of runtime feature flags, and it belongs in the product requirements before the first fleet deployment (AMD, 2024a; Chen, 2020).

1.2 Glow arch passes

Glow's **pass** pipeline is built on a two-phase strongly-typed IR, and each phase owns a different class of **optim**ization (Rotem *et al.*, 2018). The high-level IR is a static-shaped dataflow node graph where domain-specific rewrites live: merge batch-norm into convolution, change a matrix layout, eliminate a numeric rescale.

Table 1.1: Glow's two-phase IR and the optimization class each level owns (Rotem *et al.*, 2018).

IR level	Form	Optimizations
High-level	Node dataflow graph	Domain-specific (BN+conv, layout, rescale)
Low-level	Address-only instructions	Scheduling, static alloc, copy elimination
Codegen	LLVM IR	Stacked kernels, target-specific emit

IRGen then **lower**s each high-level node into one or more low-level instructions, producing an address-only IR whose operands are typed pointers to buffers; this is where memory optimizations run—instruction scheduling, static memory allocation, and copy elimination (Rotem *et al.*, 2018).

It is worth being precise about what Glow is and is not here. Glow predates the MLIR **dialect** approach that IREE and modern TVM use, and instead of an open, extensible dialect tower it hard-codes this fixed two-level design (Rotem *et al.*, 2018). The trade is less generality but a shorter, more predictable lowering path—appropriate for an edge compiler where build reproducibility and a tight memory plan matter more than supporting every research IR. For decode, the low-level IR's static memory allocation is the feature that matters: a fixed buffer plan computed at compile time is what lets a bundle run in a known footprint without an allocator in the hot path (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

What to inspect before merge. Because both IR levels are dumpable, diff the low-level IR for buffer reuse and copy elimination, and confirm that element-wise chains were stacked rather than left as separate traversals (Rotem *et al.*, 2018; Chen, 2026). A regression shows up as extra buffers or extra tensor passes, not as a wrong answer (Goodfellow *et al.*, 2016).

1.2.1 High-level rewrites that survive to decode

Glow's high-level graph rewrites were developed around convolutional networks, but the class of transform matters for decoder graphs too: merge adjacent arithmetic, eliminate redundant rescales, fold constants, and change layouts so later passes see friendly structure (Rotem *et al.*, 2018; Chen, 2020). In a small decoder, residual adds, layer-normalization scaling, and activation functions create chains that benefit from the same merge discipline (Dao *et al.*, 2022; Chen, 2026). The high-level IR is statically shaped, which makes these rewrites safe and predictable: every tensor dimension is known at compile time, so the compiler can prove whether a transform

preserves semantics (Rotem *et al.*, 2018; Davies *et al.*, 2025). Reviewers should check that the imported decoder graph actually exposes these chains, because an export that already split them into custom calls leaves the high-level passes nothing to rewrite (Chen, 2020; Davies *et al.*, 2025).

1.2.2 Static memory allocation and buffer reuse

The low-level IR's address-only form exists so memory can be planned statically: every buffer has a fixed address or a slot in a reused pool before codegen (Rotem *et al.*, 2018; Chen, 2026). The planner reuses scratch across instructions whose lifetimes do not overlap, which is how a deep model runs inside a small footprint without a hot-path allocator (Rotem *et al.*, 2018; Goodfellow *et al.*, 2016). For decode, buffer reuse is the edge form of residency: if the planner cannot overlap two intermediates, the bundle needs both simultaneously, and the sum must fit the device (Chen, 2026; Vellaisamy *et al.*, 2026). The low-level IR dump shows the reuse decisions directly, and a footprint regression after a model change can be traced to a lifetime that stopped overlapping (Rotem *et al.*, 2018; Chen, 2020). Teams should treat the static memory report as a first-class artifact, because it is the compile-time proof of the memory budget the edge device requires (AMD, 2024a; Chen, 2026).

1.2.3 Scheduling and copy elimination

Instruction scheduling in the low-level IR orders operations to reduce traffic and latency, and copy elimination removes moves whose source and destination can be unified (Rotem *et al.*, 2018; Chen, 2026). On a small core these passes matter more than on a big GPU because the memory system is smaller and every extra traversal is proportionally more expensive (Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026). The scheduling pass should also respect operator-stacking boundaries: work that will later fuse into one traversal should not be scheduled in a way that forces an intermediate materialization (Rotem *et al.*, 2018; Davies *et al.*, 2025). Diffing the low-level IR before and after a compiler change shows whether scheduling and copy elimination still behave; a regression appears as extra copy instructions or split traversals (Chen, 2026, 2020). These low-level details are where the AOT promise of predictable latency is actually delivered (Rotem *et al.*, 2018; Vellaisamy *et al.*, 2026).

1.2.4 Counting traversals

The low-level IR makes traversal count an explicit property: each instruction that reads and writes a full tensor is one pass over that tensor, and chains that fuse share one traversal (Rotem *et al.*, 2018; Chen, 2026). The compile report should count traversals per decoder layer and compare them against the minimum implied by the fused design (Chen, 2020; Vellaisamy *et al.*, 2026). A layer whose elementwise chains each traverse separately has left the stacking opportunity on the table even if every kernel is individually fast (Chen, 2026; Rotem *et al.*, 2018). This count is the edge equivalent of bytes/token because on a small memory system traversal count and bytes moved are nearly proportional (Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026). Setting a traversal budget per layer at design time gives reviewers a number to enforce and gives the compiler a target to optimize toward (Davies *et al.*, 2025; Chen, 2026).

1.2.5 MCU pipeline shape

Microcontroller decode graphs are small enough that whole-model scheduling is feasible, and Glow's fixed IR is designed for that scale (Rotem *et al.*, 2018; AMD, 2024a). The low-level instruction stream for a small decoder should be short, with most work inside stacked kernels and few control-flow branches (Rotem *et al.*, 2018; Chen, 2026). Branchy schedules that look reasonable on a host can bloat an MCU binary and introduce cache-unfriendly jumps (Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026). Static allocation on an MCU must also respect alignment and memory-mapped regions that the host planner cannot assume (AMD, 2024a; Davies *et al.*, 2025). Reviewing the emitted instruction count and branch structure for the decode loop catches these issues before device bring-up (Chen, 2020, 2026). The MCU pipeline shape is the extreme case of the AOT principle: predictability comes from keeping the compiled schedule small and fixed (Rotem *et al.*, 2018; Vellaisamy *et al.*, 2026).

1.3 Glow codegen

Glow's primary **codegen** path is its LLVM-based CPU **backend**, and its signature trick is operator stacking: instead of one kernel per element-wise operator, Glow generates LLVM IR for a single stacked kernel that performs all the chained operations during one traversal of the tensor (PyTorch / Glow contributors, 2024; Rotem *et al.*, 2018). That is the bytes/token lever in its purest form—an unstacked

chain reads and writes the tensor once per operator, while the stacked kernel touches it once (Vellaisamy *et al.*, 2026; Chen, 2026).

Because the backend is LLVM, Glow can **emit** a self-contained object file (a bundle) and cross-compile it for a different **platform** via standard LLVM target flags—for example an Arm Cortex-M7 core on an embedded board (PyTorch / Glow contributors, 2024). The decode-relevant consequence is that the bundle has a single inference entry point and no framework dispatch, so the per-token launch overhead that dominates batch-1 on a Python stack simply does not exist; the work is one compiled function call (Vellaisamy *et al.*, 2026; NVIDIA, 2024). This is the AOT counterpart to capturing a launch graph on a GPU, achieved by never having a launch loop in the first place (Chen, 2026).

There is a subtler reason operator stacking matters more at the edge than the headline FLOP count suggests. On a memory-constrained device the limiter is rarely arithmetic; it is the bandwidth between the compute core and whatever memory tier holds the activations, and every extra tensor traversal pays that bandwidth tax again (Vellaisamy *et al.*, 2026; Chen, 2026). By collapsing a chain of element-wise operators—bias add, activation function, rescale—into one traversal, Glow turns several bandwidth-bound passes into one, which on a small core is often a larger win than any single-operator speedup (Rotem *et al.*, 2018). The same logic is why the static memory plan matters: with buffers assigned at compile time, the bundle reuses a small pool of scratch memory instead of allocating per operator, so the resident footprint stays inside the device's on-chip budget and the allocator never appears in the per-token path (Goodfellow *et al.*, 2016; Davies *et al.*, 2025).

Multi-hardware delta. On a CPU or MCU the stacked kernel plus a static buffer plan is close to the best available edge decode path; on a GPU the same approach does not exploit warps or tensor cores, so Glow's CPU strength does not transfer upward (NVIDIA, 2022; SemiAnalysis, 2024). The lowering-to-primitives design is what lets a new accelerator backend reuse the high-level optimizations without reimplementing them (AMD, 2024a).

1.3.1 Stacked kernel structure

Operator stacking succeeds when the fused chain is profitable in both compute and memory terms. Elementwise work with the same iteration domain stacks cleanly; reductions and reshape-style operations break the single-traversal property unless the compiler can express them in the same loop structure (Rotem *et al.*,

2018; Chen, 2026). The generated LLVM IR should show one loop nest reading the input tensor once and applying the whole chain, with temporaries in registers or a small scratch region (PyTorch / Glow contributors, 2024; Vellaisamy *et al.*, 2026). For decoder chains of bias, activation, scaling, and residual adds, that single traversal is the whole win, and splitting it re-introduces the bandwidth tax the edge device cannot afford (Dao *et al.*, 2022; Chen, 2026). Reviewers should therefore inspect the emitted IR for loop count per tensor and treat a chain that materialized intermediate tensors as a fusion failure (Rotem *et al.*, 2018; Davies *et al.*, 2025).

1.3.2 Cross-compilation and bundle boundaries

Glow emits bundles for the host or for a cross-compiled target through LLVM target flags, and the bundle boundary is a deployment decision (PyTorch / Glow contributors, 2024; Rotem *et al.*, 2018). One bundle per model keeps the host call simple but forces the whole model's schedule to be fixed together; per-layer or per-bucket bundles give the host more control at the cost of more entry points and buffer handoffs (Davies *et al.*, 2025; Chen, 2026). For a decoder with multiple context buckets, separate bundles per bucket let the quantized ranges and memory plans differ per bucket while the host selects the right entry (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). The handoffs between bundles must reuse preallocated buffers, because a bundle boundary that allocates on the host reintroduces the allocator churn the AOT design removes (Chen, 2026; Davies *et al.*, 2025). The artifact manifest should list every bundle, its target, its memory plan, and the bucket it serves (Chen, 2020; Rotem *et al.*, 2018).

1.3.3 Counting calls per token

The edge analog of launches/token is the number of compiled entry calls and buffer handoffs the host performs for one decode token (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). A single whole-model bundle reduces that count to one call; a per-bucket split adds a bucket-selection branch; a per-layer split multiplies calls by layer depth (Rotem *et al.*, 2018; Chen, 2026). On a microcontroller, each call has a measurable cost in stack setup, buffer pointer marshaling, and possible cache effects, so the call count belongs in the benchmark (Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026). The right granularity balances call count against memory: a whole-model bundle may need more SRAM than the device has, forcing a split even though the single call would be cheaper (AMD, 2024a; Chen, 2026). The design review should therefore state the call-count target and the memory budget

together, because optimizing one without the other fails on edge hardware (Davies *et al.*, 2025; Rotem *et al.*, 2018).

1.3.4 Vectorization on small cores

The LLVM backend's vectorization matters on edge CPUs that support SIMD and on microcontrollers where only scalar code fits (PyTorch / Glow contributors, 2024; Goodfellow *et al.*, 2016). A stacked kernel's loop should vectorize over the contiguous tensor dimension so each traversal also uses the widest available loads and arithmetic (Rotem *et al.*, 2018; Davies *et al.*, 2025). Cross-compiling the same bundle to a core without SIMD should degrade gracefully rather than fail, because the deployment target may be narrower than the build target (PyTorch / Glow contributors, 2024; Chen, 2020). The compile report should state the vector width actually selected for each stacked kernel so a perf regression can be traced to a target flag change (Chen, 2026; Vellaisamy *et al.*, 2026). Operator stacking and vectorization are complementary: stacking removes passes, and vectorization makes each remaining pass cheaper (Rotem *et al.*, 2018; Chen, 2026).

1.4 Glow tuning

Glow's most distinctive tuning mechanism is **profile**-guided quantization, and it is profile-first by construction (Rotem *et al.*, 2018). The flow is two-phase: statically instrument the network with profiling nodes, run inference on representative data to record the numeric range of each activation, then recompile using that profile to convert the float network into int8 islands of computation, fusing away rescales between nodes (PyTorch / Glow contributors, 2024). On a memory-bound edge target this is a direct bytes/token win—int8 activations and weights move a quarter of the bytes of float32—and it reduces the model footprint enough to fit parts that float32 never would (Vellaisamy *et al.*, 2026; Chen, 2026).

The **pitfall** is the profiling data and the operating point. The recorded ranges are only as representative as the inputs you profiled, so a profile captured on prefill-style batches can mis-estimate the activation ranges seen during long-context batch-1 decode, producing saturation or precision loss exactly where it hurts output quality (Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Two disciplines follow: profile on decode-representative sequences at the context lengths you actually serve, and re-profile when the workload shifts, because the AOT bundle freezes the quantization decision at compile time. As always the right configuration is **hardware**-specific—an MCU may force int8 everywhere while a host CPU can

keep sensitive layers in float—so the tuned bundle for one target does not transfer to another (AMD, 2024b; Davies *et al.*, 2025).

1.4.1 Quantization profile discipline

Profile-guided quantization turns a compiler feature into a data-quality problem: the recorded activation ranges determine both speed and output quality (Rotem *et al.*, 2018; Davies *et al.*, 2025). The profiling corpus must cover the context lengths, attention patterns, and temperature behavior the served workload actually exhibits, because saturation in a rare long-context request is invisible in a profile of short prompts (Kwon *et al.*, 2023; Dao *et al.*, 2022). Per-channel and per-tensor range choices should be explicit in the compile report so a precision regression can be traced to the layer whose range was mis-calibrated (Chen, 2020; Davies *et al.*, 2025). Re-profiling must be triggered by workload shifts, not just model changes, since the same weights can produce different activation ranges under different serving mixes (Chen, 2026; Davies *et al.*, 2025). Teams that skip profile discipline get int8 bundles that pass unit tests and fail on real traffic (Vellaisamy *et al.*, 2026; Rotem *et al.*, 2018).

1.4.2 Islands, rescales, and fusion

Quantization in Glow converts parts of the graph into int8 islands separated by rescale operations, and the compiler's goal is to eliminate as many rescales as possible so the islands fuse into single-traversal kernels (Rotem *et al.*, 2018; PyTorch / Glow contributors, 2024). Every surviving rescale is a potential full-tensor traversal, so the island boundary design determines how much of the int8 bytes win survives into actual traffic reduction (Chen, 2026; Vellaisamy *et al.*, 2026). Decoder layers have natural island boundaries around matmuls, with elementwise chains inside islands and rescales at the numerical edges (Dao *et al.*, 2022; Davies *et al.*, 2025). The compile report should show island structure per layer, and a chain that splits into multiple int8 islands deserves the same scrutiny as an unfused chain in the datacenter compilers (Rotem *et al.*, 2018; Chen, 2020). Reviewers should ask where each rescale lives and whether it could have been folded, because each removable rescale is a full bandwidth pass on the edge (Chen, 2026; Davies *et al.*, 2025).

1.4.3 Target-specific precision policies

One float model should not quantize identically on every device. A microcontroller may need int8 everywhere to fit memory, while a host CPU can keep numerically sensitive layers in float and quantize only the bandwidth-bound matmuls (Rotem *et al.*, 2018; Goodfellow *et al.*, 2016; AMD, 2024a). Glow's compile-time flow supports target-specific policies because the precision plan is part of the AOT bundle, not a runtime decision (PyTorch / Glow contributors, 2024; Rotem *et al.*, 2018). The policy should be derived from the residency budget and the quality requirement together: choose per layer the precision that fits the device and meets the error budget, then verify the bundle numerically (Dao *et al.*, 2022; Davies *et al.*, 2025). A policy table per target belongs in the release manifest so a precision change is a reviewable diff rather than an undocumented flag (Chen, 2020, 2026). Porting a bundle between targets should force a policy review, never a silent copy (Rotem *et al.*, 2018; Vellaisamy *et al.*, 2026).

1.4.4 Benchmarking the bundle

The bundle benchmark should measure what the device will actually do per token: the compiled call, the buffer handoffs, and the memory traffic (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). It should run on the target or an accurate simulator because host benchmarks hide cache, power, and instruction-set differences (Goodfellow *et al.*, 2016; Chen, 2026). The report should include latency, energy if measurable, peak memory from the static plan, and output quality error against the float reference (Rotem *et al.*, 2018; Davies *et al.*, 2025). Each candidate configuration (precision policy, island layout, bundle boundary) should produce one benchmark row so the trade becomes visible (Chen, 2020; Vellaisamy *et al.*, 2026). Edge decode teams should treat this benchmark as the acceptance gate, because no compiler flag matters more than the measured behavior on the actual device (Chen, 2026; Rotem *et al.*, 2018).

1.4.5 Numerical error budgets across layers

Quantization error compounds through a decoder, so the per-layer range decisions should be governed by an end-to-end error budget rather than by local accuracy checks (Dao *et al.*, 2022; Davies *et al.*, 2025). Layers that amplify error, such as attention accumulations and normalization, deserve wider ranges or higher precision even when their bytes could be reduced (Chen, 2026; Dao *et al.*, 2022). The budget should be verified on decode-representative outputs, comparing the

int8 bundle against the float reference with a bounded-error metric (Rotem *et al.*, 2018; Davies *et al.*, 2025). When the end-to-end error exceeds the budget, the trace locates the dominating layer and the fix targets that layer only (Chen, 2020; Rotem *et al.*, 2018). Teams that tune per-layer precision without an end-to-end budget will chase accuracy regressions one layer at a time after every workload shift (Vellaisamy *et al.*, 2026; Chen, 2026). Run the bounded-error check at every shipped context bucket and request type, because rare quality failures hide in the tail of the workload distribution (Davies *et al.*, 2025; Dao *et al.*, 2022). A bundle that passes average-error gates can still violate the product quality SLO on the longest prompts or the least common routing pattern, because quantization error compounds with both attention depth and sequence length (Kwon *et al.*, 2023; Chen, 2026).

1.5 Glow case study

Take a small decoder model compiled to an edge bundle as the **case**, and **benchmark** the int8 AOT bundle against a float interpreter at the decode operating point (Rotem *et al.*, 2018; Vellaisamy *et al.*, 2026). The reading matches the book's thesis from the edge side: the win is not a faster matmul but the removal of dispatch overhead and HBM/DRAM round trips—the stacked kernel keeps element-wise work in one traversal and the static buffer plan avoids allocator churn, so bytes/token and launches/token both drop while the arithmetic is unchanged (Chen, 2026; Dao *et al.*, 2022). The quantization adds a second bytes/token win by shrinking every activation to int8 (PyTorch / Glow contributors, 2024). The arithmetic is direct: a decoder layer that moves float32 activations spends four bytes per element, and converting to int8 cuts that to one, so on a bandwidth-bound edge core the memory traffic for those activations drops by roughly four times before any kernel fusion is counted (Vellaisamy *et al.*, 2026). That is the dominant reason int8 bundles are recommended for memory-constrained deployment: the speedup comes from moving fewer bytes, not from faster multiplies.

The comparison also exposes a limit worth stating plainly. An interpreter dispatches each operator through a generic runtime, paying overhead per node that, at batch-1, has no batch to amortize against—which is precisely the per-token tax Chapter 1 measured on the Python stack (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). The AOT bundle removes that tax structurally by compiling the whole network into one function, so the speedup over an interpreter widens as the model gets deeper and the per-operator overhead would otherwise accumulate (Rotem *et al.*, 2018; Chen, 2026).

A second lesson from the case is about debuggability under quantization. Because Glow emits debug information in terms of its own IR rather than raw machine code, a precision regression introduced by an int8 conversion can be traced back to the specific node whose recorded range was too tight, rather than guessing from a wrong output (Rotem *et al.*, 2018). For an edge decode deployment that traceability is the difference between a quick re-profile of one layer and a blind re-quantization of the whole model, and it is the kind of compile-time visibility the AOT path buys that a runtime interpreter cannot (PyTorch / Glow contributors, 2024; Davies *et al.*, 2025).

The **cross-hardware** conclusion is where Glow's scope is honest. The same bundle source cross-compiles from x86 to Cortex-M, but the two targets have wildly different residency budgets, so the fusion and quantization plan that fits the host may overflow the MCU's SRAM (Rotem *et al.*, 2018; AMD, 2024a). And none of Glow's CPU/edge strength competes with a CUDA-graph-captured Triton or IREE path on a datacenter GPU—which is the correct division of labor, not a deficiency (NVIDIA, 2024; SemiAnalysis, 2024). **YiRage** reflects exactly this: it uses Glow-style AOT lowering and operator stacking for edge and CPU targets while routing datacenter-GPU decode to its GPU backends, and carries the Chapter 2 residency rules as checkable IR metadata so an edge bundle that would overflow on-chip memory is rejected at compile time (Chen and YiRage Contributors, 2026).

1.5.1 Worked bundle worksheet

The edge case study produces a worksheet with the same rows as its datacenter counterparts, translated to edge units: the interpreter baseline's per-token calls and memory passes, the AOT bundle's compiled call count and traversal count, and the int8 bundle's bytes-per-element reduction (Rotem *et al.*, 2018; Vellaisamy *et al.*, 2026; Chen, 2026). Each row should list peak SRAM from the static plan so the memory claim is part of the benchmark (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). The win attribution should separate the three levers: operator stacking removes passes, static allocation removes allocator churn, and quantization shrinks bytes per element (Rotem *et al.*, 2018; Chen, 2026). A bundle that only wins on one lever is still valuable but should be labeled accordingly, because the three levers fail differently on different workloads (Chen, 2020; Davies *et al.*, 2025). Publishing the worksheet with each bundle release keeps the edge team's claims reproducible (Vellaisamy *et al.*, 2026; Rotem *et al.*, 2018).

1.5.2 Precision regression tracing

When an int8 bundle changes output quality, the trace starts at the layer whose profile range was tightest relative to the observed traffic (Rotem *et al.*, 2018; Davies *et al.*, 2025). Glow's IR-level debug information maps the wrong output back to the node and its recorded range, which turns a quality incident into a targeted re-profile (PyTorch / Glow contributors, 2024; Rotem *et al.*, 2018). The workflow is to compare the profile ranges against the ranges in the failing traffic, widen the mis-calibrated layer, recompile, and re-run the quality gate (Chen, 2020; Davies *et al.*, 2025). The fix should never be a global precision downgrade, because that surrenders the bytes win on layers that were calibrated correctly (Chen, 2026; Vellaisamy *et al.*, 2026). This traceability is the practical payoff of compile-time visibility, and it is a capability the runtime interpreter cannot offer because it has no recorded compilation state (Rotem *et al.*, 2018; Chen, 2026).

1.5.3 Edge host orchestration

The compiled bundle still needs a small host program to manage sensors, request scheduling, tokenization, and output handling around it (Kwon *et al.*, 2023; Davies *et al.*, 2025). That host code is part of the decode path, so it must be measured with the bundle: a host loop that copies buffers or allocates per token erases the AOT advantage (Chen, 2026; Vellaisamy *et al.*, 2026). Buffer ownership should be fixed at startup, with the bundle reading and writing preallocated memory (Rotem *et al.*, 2018; Davies *et al.*, 2025). If the edge device runs several bundles or models, the host scheduler owns their interleaving and must respect the static memory plan of each (AMD, 2024a; Chen, 2026). The boundary between compiled code and host logic is the edge analog of the eager-hull design from Chapter 12, and it deserves the same telemetry so per-token time is attributed correctly (NVIDIA, 2024; Vellaisamy *et al.*, 2026).

1.5.4 Failure autopsies for edge decode

Three signatures dominate Glow edge incidents. A bundle that fits in the compile report but faults on device usually means the static plan assumed more memory than the runtime left available, so the plan must be checked against the real allocator layout (Rotem *et al.*, 2018; AMD, 2024a). A quality regression after quantization traces to profile ranges, fixed by widening the mis-calibrated layers (Rotem *et al.*, 2018; Davies *et al.*, 2025). A latency regression after a host change usually means the orchestration around the bundle started copying or allocating

per token (Chen, 2026; Vellaisamy *et al.*, 2026). Each autopsy starts from the artifact manifest and the compile reports, which makes it a diff against the last known-good bundle (Chen, 2020, 2026). Teams that archive reports with bundles turn every incident into a bounded check (Rotem *et al.*, 2018; Vellaisamy *et al.*, 2026).

1.5.5 Quantization-first versus float-first bring-up

Bring-up ordering changes how quickly a team learns whether a model fits an edge device (Rotem *et al.*, 2018; Davies *et al.*, 2025). A float-first bring-up isolates precision and scheduling problems before quantization adds its own variables, but it may conclude the model does not fit when an int8 version would (Chen, 2026; Vellaisamy *et al.*, 2026). A quantization-first bring-up measures the realistic footprint early, but it risks conflating quantization errors with schedule errors when something fails (Dao *et al.*, 2022; Davies *et al.*, 2025). The recommended path is a quick float feasibility compile to confirm the graph imports and the memory plan roughly fits, followed immediately by a profile-guided int8 compile at the served workload (Rotem *et al.*, 2018; Davies *et al.*, 2025). Each step publishes the same worksheet fields, so the team always knows whether the current blocker is structure, schedule, memory, or precision (Chen, 2020, 2026). This ordering converts edge bring-up from a debugging spiral into a sequence of gates (Vellaisamy *et al.*, 2026; Rotem *et al.*, 2018).

1.6 Key Takeaways

- **Glow is the edge/CPU compiler, not the GPU-decode tool.** Its LLVM CPU backend and AOT bundles target memory-constrained devices, and lowering to a few linear-algebra primitives makes new accelerator backends cheap (Rotem *et al.*, 2018; PyTorch / Glow contributors, 2024). For datacenter GPU decode, prefer the CUDA-graph paths of Chapters 16–18 (NVIDIA, 2024).

- **Operator stacking is the bytes/token lever.** Glow's stacked kernel performs chained element-wise work in a single tensor traversal instead of one kernel per operator, removing redundant DRAM round trips (PyTorch / Glow contributors, 2024; Chen, 2026). Confirm chains were stacked by diffing the low-level IR (Vellaisamy *et al.*, 2026).

- **AOT bundles remove the launch loop entirely.** A self-contained object

file with a single inference entry point has no framework dispatch, so the batch-1 launch overhead that dominates a Python stack does not exist (Rotem *et al.*, 2018; Vellaisamy *et al.*, 2026). It is the edge counterpart to capturing a launch graph (NVIDIA, 2024).

- **Profile-guided quantization needs decode-representative data.** Glow records activation ranges by running inference, then recompiles to int8; a profile captured on the wrong shape mis-estimates ranges and loses precision where decode needs it (Rotem *et al.*, 2018; Kwon *et al.*, 2023). Profile at served context lengths and re-profile on workload shift (Davies *et al.*, 2025).
- **The two-phase IR trades generality for a predictable footprint.** A fixed high-level graph plus low-level address-only IR (with static memory allocation) is less general than an MLIR dialect tower but yields a known memory plan—the right trade for the edge (Rotem *et al.*, 2018; Goodfellow *et al.*, 2016). Tuned bundles are per-target (AMD, 2024b).

Practical decision rules. Four rules summarize Glow for an edge decode review. First, set a traversal-count target per tensor chain and verify stacking in the low-level IR, because each traversal is a bandwidth pass on a memory-constrained core (Rotem *et al.*, 2018; Chen, 2026). Second, treat the static memory plan as a hard contract: the plan plus the host's reservations must fit the device before any benchmark matters (Goodfellow *et al.*, 2016; AMD, 2024a). Third, derive every quantization decision from a decode-representative profile and keep the per-layer ranges in the release manifest (Rotem *et al.*, 2018; Davies *et al.*, 2025). Fourth, measure the whole host-plus-bundle path on the target device, including calls, handoffs, and energy where available (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Teams that apply these rules get predictable edge decode; teams that skip them get a bundle that compiled cleanly and failed in the field (Chen, 2026; Rotem *et al.*, 2018).

1.7 Conclusion

Glow shows that the residency-first thesis is not GPU-specific: at the edge, lowering to a few primitives, stacking operators into single-traversal kernels, statically planning memory, and quantizing under a recorded profile are the moves that drop bytes/token and erase launch overhead (Rotem *et al.*, 2018; PyTorch / Glow contributors, 2024). Its deliberate scope—CPU, accelerators, and memory-constrained devices rather than datacenter GPUs—makes it the right backend for

one slice of the hardware matrix and the wrong one for another, which is exactly the routing decision YiRage encodes as IR metadata (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026). The broader point is that a lightweight compiler is not a lesser one: by giving up generality it buys a predictable memory plan and a dispatch-free runtime, and those are precisely the properties that make decode latency stable on a small device where a heavyweight stack would never fit (Rotem *et al.*, 2018; Davies *et al.*, 2025). Having now surveyed established compiler surfaces from TVM through Glow, Chapter **??** turns to Mirage and the emerging wave of AI compilers that push these ideas—whole-program scheduling, superoptimization, and residency-aware fusion—past what today's production stacks ship (Chen, 2026; Goodfellow *et al.*, 2016).

Bibliography

AMD (2024a). Ai engine technology. `https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html`. 2, 3, 4, 6, 7, 8, 9, 12, 14, 15, 17

AMD (2024b). AMD XDNA architecture. `https://www.AMD.com/en/technologies/XDNA.html`. 2, 11, 17

Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18

Chen, X. (2020). Compiler-based hardware profiling for software-defined architecture. 2, 3, 4, 5, 6, 7, 9, 10, 11, 12, 13, 14, 15, 16

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. `https://github.com/chenxingqiang/YiRage`. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 2, 14, 18

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*. 2, 5, 9, 11, 12, 13, 16

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefill nodes. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 1, 2, 3, 5, 6, 7, 8, 9, 10, 12, 14, 17, 18

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 9, 10, 11, 13, 15, 17

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 8

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 1, 2, 8, 14, 15, 16, 17

PyTorch / Glow contributors (2024). Glow: Compiler for neural network hardware accelerators. `https://github.com/pytorch/glow`. LLVM-based CPU backend with operator stacking; `model-compiler` AOT bundles; profile-guided quantization; cross-compile to edge/MCU. 1, 2, 3, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17

Rotem, N., Fix, J., Abdulrasool, S., Catron, G., Deng, S., Dzhabarov, R., Gibson, N., Hegeman, J., Lele, M., Levenstein, R., Montgomery, J., Maher, B., Nadathur, S., Olesen, J., Park, J., Rakhov, A., and Smelyanskiy, M. (2018). Glow: Graph lowering compiler techniques for neural networks. *arXiv preprint arXiv:1805.00907*. Two-phase strongly-typed IR (high-level graph + low-level address-only); lowering to linear-algebra primitives; profile-guided quantization; AOT bundles. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18

SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 8, 14

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18

Index

Glow, 1

Operator stacking, 1

Profile-guided quantization, 1